

D1: Working with AI

Name: Alston Liow Gun You

Student ID: 37440985

Date: 02 September 2026

Overview

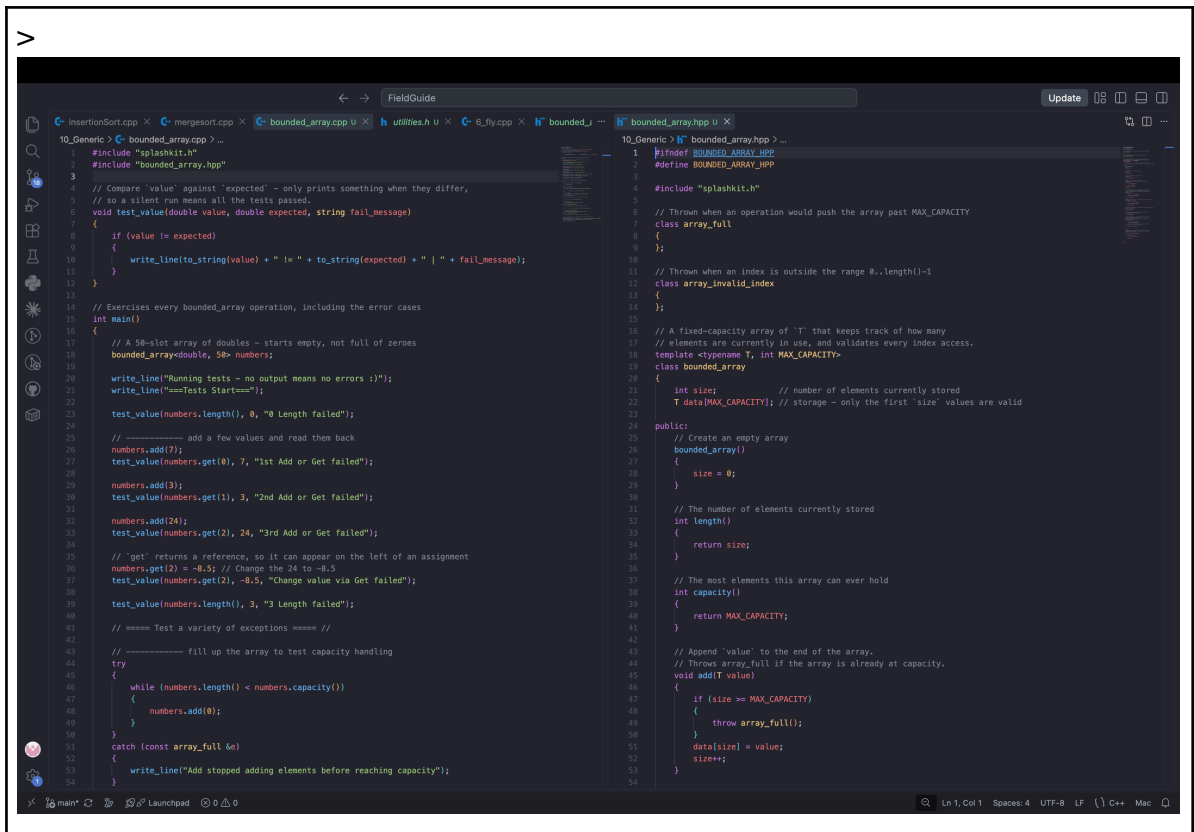
This task is an opportunity for you to demonstrate your ability to learn programming topics, and to explore how to work with AI on coding projects.

Do the following:

Part 1: Learning about generics and operators

Before we use AI, you need to learn some new programming tools. The end goal is for you to build a dynamic array using AI, but for you to be in control of what the AI does and have the necessary level of understanding to critically evaluate the code it produces for you. To do this, you need first to learn about generics and operators.

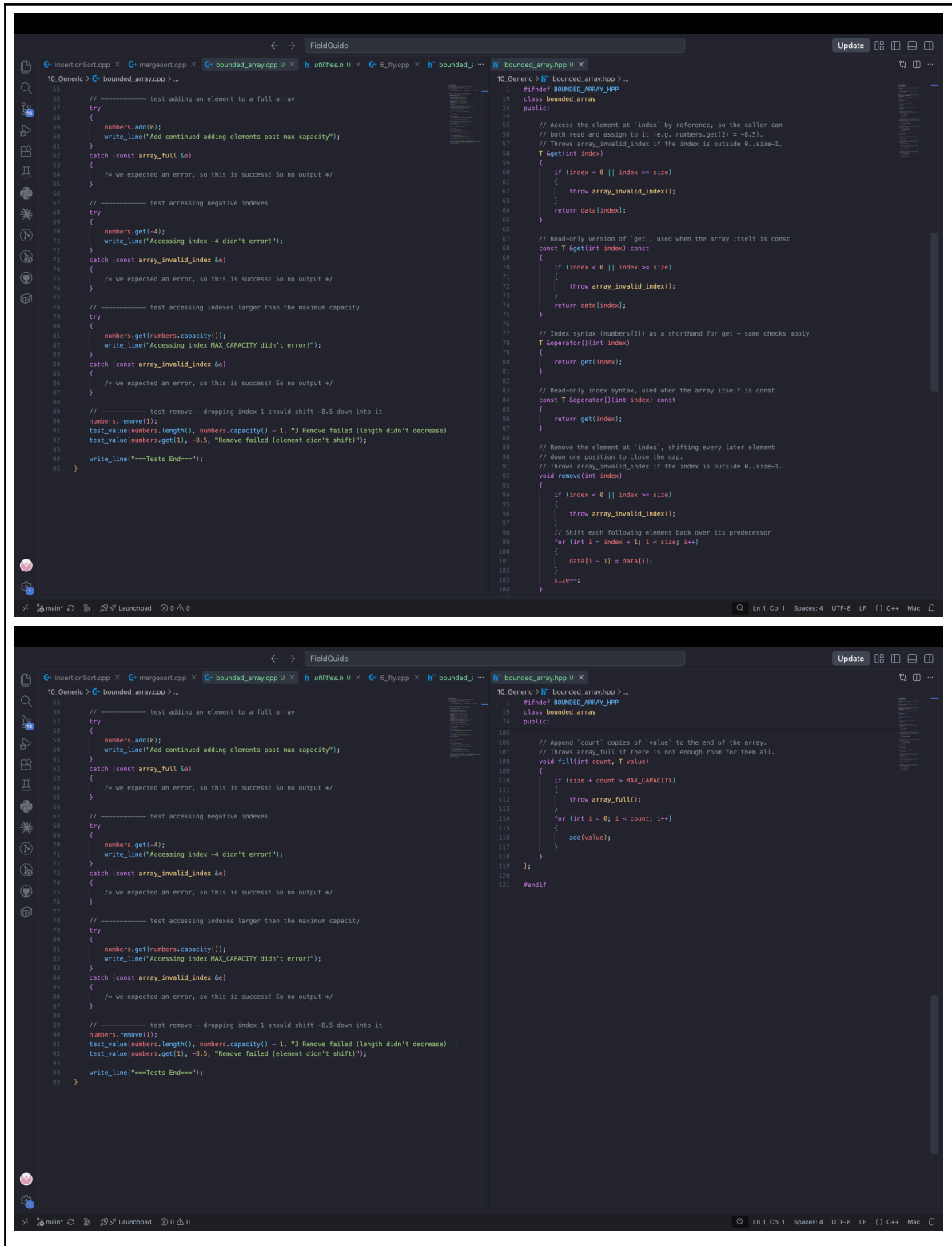
- Everything you need is in the [Generics & Operators](#) chapter of our [Programmer's Field Guide](#). Make use of the [guided tours](#) to explore these concepts.
- Add a screenshot showing you were able to create your own generic array. You can use the [generic array tour](#).



```

10_Generic.cpp
1 #include "splashkit.h"
2 #include "bounded_array.hpp"
3
4 // Compare 'value' against 'expected' - only prints something when they differ,
5 // so a silent run means all the tests passed.
6 void test_value(double value, double expected, string fail_message)
7 {
8     if (value != expected)
9     {
10         write_line(to_string(value) + " != " + to_string(expected) + " | " + fail_message);
11     }
12 }
13
14 // Exercises every bounded_array operation, including the error cases
15 int main()
16 {
17     // A 50-slot array of doubles - starts empty, not full of zeroes
18     bounded_array<double> 50= numbers;
19
20     write_line("Running tests - no output means no errors!");
21     write_line("====Tests Start====");
22
23     test_value(numbers.length(), 0, "0 Length failed");
24
25     // ===== add a few values and read them back
26     numbers.add(7);
27     test_value(numbers.get(0), 7, "1st Add or Get failed");
28
29     numbers.add(3);
30     test_value(numbers.get(1), 3, "2nd Add or Get failed");
31
32     numbers.add(24);
33     test_value(numbers.get(2), 24, "3rd Add or Get failed");
34
35     // 'get' returns a reference, so it can appear on the left of an assignment
36     numbers.get(2) = -8.5; // Change the 24 to -8.5
37     test_value(numbers.get(2), -8.5, "Change value via Get failed");
38
39     test_value(numbers.length(), 3, "3 Length failed");
40
41     // ===== Test a variety of exceptions =====
42
43     // ===== fill up the array to test capacity handling
44     try
45     {
46         while (numbers.length() < numbers.capacity())
47         {
48             numbers.add(0);
49         }
50     }
51     catch (const array_full &e)
52     {
53         write_line("Add stopped adding elements before reaching capacity");
54     }
55 }
56
10_Generic.h
1 #ifndef BOUNDED_ARRAY_HPP
2 #define BOUNDED_ARRAY_HPP
3
4 #include "splashkit.h"
5
6 // Thrown when an operation would push the array past MAX_CAPACITY
7 class array_full
8 {
9 };
10
11 // Thrown when an index is outside the range 0..length()-1
12 class array_invalid_index
13 {
14 };
15
16 // A fixed-capacity array of 'T' that keeps track of how many
17 // elements are currently in use, and validates every index access.
18 template <typename T, int MAX_CAPACITY>
19 class bounded_array
20 {
21     int size; // number of elements currently stored
22     T data[MAX_CAPACITY]; // storage - only the first 'size' values are valid
23
24 public:
25     // Create an empty array
26     bounded_array()
27     {
28         size = 0;
29     }
30
31     // The number of elements currently stored
32     int length()
33     {
34         return size;
35     }
36
37     // The most elements this array can ever hold
38     int capacity()
39     {
40         return MAX_CAPACITY;
41     }
42
43     // Append 'value' to the end of the array.
44     // Throws array_full if the array is already at capacity.
45     void add(T value)
46     {
47         if (size == MAX_CAPACITY)
48         {
49             throw array_full();
50         }
51         data[size] = value;
52         size++;
53     }
54 }

```



```

10_Generic.cpp
// ----- test adding an element to a full array
try
{
    numbers.add(0);
    write_line("Add continued adding elements past max capacity");
    catch (const array_full &e)
    {
        /* we expected an error, so this is success! So no output */
    }

// ----- test accessing negative indexes
try
{
    numbers.get(-4);
    write_line("Accessing index -4 didn't error!");
    catch (const array_invalid_index &e)
    {
        /* we expected an error, so this is success! So no output */
    }

// ----- test accessing indexes larger than the maximum capacity
try
{
    numbers.get(numbers.capacity());
    write_line("Accessing index MAX_CAPACITY didn't error!");
    catch (const array_invalid_index &e)
    {
        /* we expected an error, so this is success! So no output */
    }

// ----- test remove - dropping index 1 should shift -8.5 down into it
numbers.remove(1);
test_value(numbers.length(), numbers.capacity() - 1, "3 Remove failed (length didn't decrease)");
test_value(numbers.get(1), -8.5, "Remove failed (element didn't shift)");
write_line("==Tests End==");
}

bounded_array.hpp
1 #ifndef BOUNDED_ARRAY_HPP
2 #define BOUNDED_ARRAY_HPP
3
4 class bounded_array
5 {
6 public:
7     // Access the element at 'index' by reference, so the caller can
8     // both read and assign to it (e.g. numbers.get(2) = -8.5).
9     // Throws array_invalid_index if the index is outside 0..size-1.
10    T& get(int index)
11    {
12        if (index < 0 || index >= size)
13        {
14            throw array_invalid_index();
15        }
16        return data[index];
17    }
18
19    // Read-only version of 'get', used when the array itself is const
20    const T& get(int index) const
21    {
22        if (index < 0 || index >= size)
23        {
24            throw array_invalid_index();
25        }
26        return data[index];
27    }
28
29    // Index syntax (numbers[2]) as a shorthand for get - same checks apply
30    T& operator[](int index)
31    {
32        return get(index);
33    }
34
35    // Read-only index syntax, used when the array itself is const
36    const T& operator[](int index) const
37    {
38        return get(index);
39    }
40
41    // Remove the element at 'index', shifting every later element
42    // down one position to close the gap.
43    // Throws array_invalid_index if the index is outside 0..size-1.
44    void remove(int index)
45    {
46        if (index < 0 || index >= size)
47        {
48            throw array_invalid_index();
49        }
50        // Shift each following element back over its predecessor
51        for (int i = index + 1; i < size; i++)
52        {
53            data[i - 1] = data[i];
54        }
55        size--;
56    }
57
58    // Append 'count' copies of 'value' to the end of the array.
59    // Throws array_full if there is not enough room for them all.
60    void fill(int count, T value)
61    {
62        if (size + count > MAX_CAPACITY)
63        {
64            throw array_full();
65        }
66        for (int i = 0; i < count; i++)
67        {
68            add(value);
69        }
70    }
71
72 #endif

```

- Extend your glossary with a page on generics, and a separate page on overloading operators.
- We recommend discussing your progress with the teaching team at this point. No need to submit to OnTrack - just speak with us in class.

Tip: Distinction level task

We want you to take greater responsibility for exploring these advanced topics yourself. Feel free to explore a range of learning resources, but make sure to bring it back and apply our coding convention to what you produce.

Part 2: Learning about pointers

The next part of this puzzle is pointers. You need pointers before you can work with dynamic memory. These will test your ability to think clearly about how your programs are running, what is in memory, and where.

- Everything you need is in the [Pointers & Lists](#) chapter of our [Programmer's Field Guide](#).
- Add a screenshot showing you were able to create your own simple linked list. You can use the [linked lists tour](#) (multiple pages). We suggest getting at least adding, iterating, and deleting nodes working. Though, a good test of your capabilities is to add an `insert` operation as well.

FieldGuide > 10_Pointer&List > N linked_list.hpp > ...

```

1 #ifndef LINKED_LIST_HPP
2 #define LINKED_LIST_HPP
3
4 #include "splashkit.h"
5
6 /**
7  * A node is a class that contains a pointer to the next node,
8  * and a data value
9  *
10  * @param T the type of the data will be stored in the node
11  */
12 template <typename T>
13 class node
14 {
15 public:
16     // last node when next is nullptr
17     node* next;
18     // the value this node stores
19     T data;
20 };
21
22 /**
23  * Linked list is a class that contains a pointer to the first node,
24  * and the last node
25  *
26  * @param T is type of data
27  */
28 template <typename T>
29 class linked_list
30 {
31 public:
32     // head of the chain, nullptr when the list is empty
33     node<T>* first;
34     // tail of the chain, kept so add_node does not have to walk the whole list
35     node<T>* last;
36
37     /**
38      * Create an empty list
39      */
40     linked_list()
41     {
42         // no nodes yet, so there is no head and no tail
43         first = nullptr;
44         last = nullptr;
45     }
46
47     /**
48      * Add new node to the end of the list
49      *
50      * @param data The data to add
51      */
52     node<T>* add_node(T data)
53     {
54         // declare a pointer for the new node (not yet pointing to valid memory)
55         node<T>* new_node;
56         // allocate a new node on the heap and store its address in new_node
57         new_node = new node<T>();
58
59         // fill in the new node's data
60         new_node->data = data;
61         // this node will be the new tail, so it has no next node yet
62         new_node->next = nullptr;
63
64         // case 1: list is currently empty
65         if (first == nullptr)
66         {
67             // new node is both the first and last node
68             first = new_node;
69             last = new_node;
70         }
71         // case 2: list already has at least one node
72         else
73         {
74             // link the current last node to the new node
75             last->next = new_node;
76             // update last to point to the new tail
77             last = new_node;
78         }
79
80         return new_node;
81     }
82
83     /**
84      * Find and return the node previous to 'target_node'
85      *
86      * @param target_node
87      */
88     node<T>* find_previous_node(node<T>* target_node)
89     {
90         // the head has nothing before it, so there is no previous node
91         if (target_node == first)
92         {
93             return nullptr;
94         }
95
96         // walk the list from the front, one node at a time
97         node<T>* current = first;
98         while (current != nullptr)
99         {
100             // if the node after current is the one we want, current is the answer
101             if (current->next == target_node)
102             {
103                 return current;
104             }
105             // step forward along the chain
106             current = current->next;
107         }
108
109         // reached the end without finding target_node, so it isn't in this list
110         throw string("find_previous_node search failed: node not in list.");
111     }
112
113     /**
114      * Clear all node
115      */
116     void clear()
117     {
118         node<T>* current = first;
119         while (current != nullptr)
120         {
121             // remember the next node BEFORE deleting current,
122             // otherwise current->next is gone and we lose the rest of the list
123             node<T>* next = current->next;
124             // free the heap memory for this node
125             delete current;
126             // move on to the node we saved
127             current = next;
128         }
129         // every node is gone, so the list is now empty
130         first = nullptr;
131         last = nullptr;
132     }
133
134     /**
135      * remove the specific node from the list
136      *
137      * @param del_node
138      */
139     void remove(node<T>* del_node)
140     {
141         node<T>* previous_node = nullptr;
142
143         // case 1: deleting the head, so the second node becomes the new first
144         if (first == del_node)
145         {
146             first = del_node->next;
147             // nothing points to del_node, so there is no previous node
148             previous_node = nullptr;
149         }
150         // case 2: deleting a node in the middle or at the end
151         else
152         {
153             // find the node that currently points to del_node
154             previous_node = find_previous_node(del_node);
155             // unlink del_node by pointing previous straight at del_node's next
156             previous_node->next = del_node->next;
157         }
158
159         // if we removed the tail, the previous node becomes the new tail
160         // (nullptr when the list is now empty)
161         if (last == del_node)
162         {
163             last = previous_node;
164         }
165
166         // now that nothing points to it, free its heap memory
167         delete del_node;
168     }
169
170     /**
171      * Insert a node into a list, after the previous list
172      *
173      * @param in_node
174      * @param previous_node
175      */
176     void insert(node<T>* in_node, node<T>* previous_node)
177     {
178         // the new node takes over whatever previous_node used to point at
179         in_node->next = previous_node->next;
180         // previous_node now points at the new node, splicing it into the chain
181         previous_node->next = in_node;
182
183         // if we inserted after the old tail, the new node is the new tail
184         if (last == previous_node)
185         {
186             last = in_node;
187         }
188     }
189 }

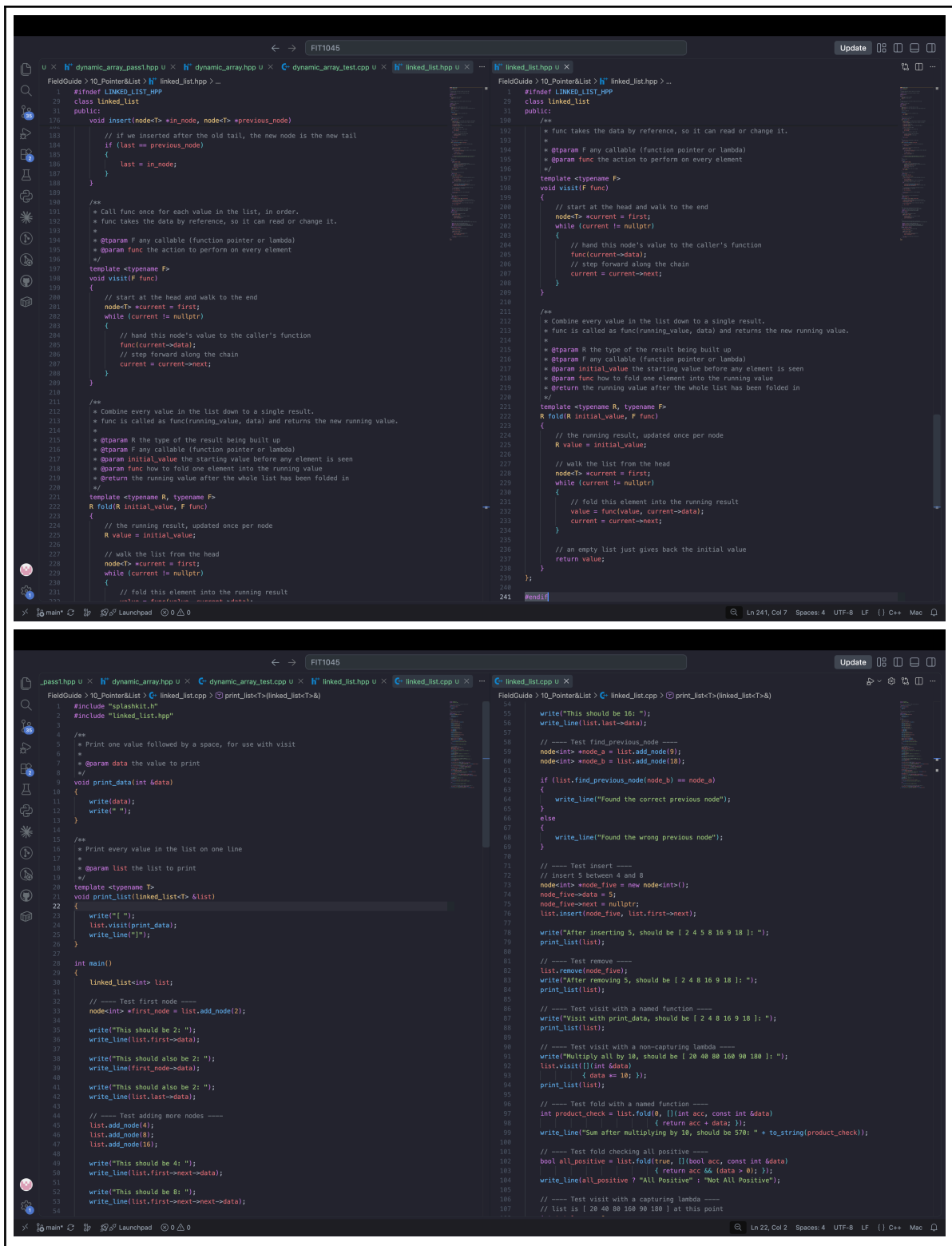
```

FieldGuide > 10_Pointer&List > N linked_list.hpp > ...

```

1 #ifndef LINKED_LIST_HPP
2 #define LINKED_LIST_HPP
3
4 #include "splashkit.h"
5
6 /**
7  * A node is a class that contains a pointer to the next node,
8  * and a data value
9  *
10  * @param T the type of the data will be stored in the node
11  */
12 template <typename T>
13 class node
14 {
15 public:
16     // last node when next is nullptr
17     node* next;
18     // the value this node stores
19     T data;
20 };
21
22 /**
23  * Linked list is a class that contains a pointer to the first node,
24  * and the last node
25  *
26  * @param T is type of data
27  */
28 template <typename T>
29 class linked_list
30 {
31 public:
32     // head of the chain, nullptr when the list is empty
33     node<T>* first;
34     // tail of the chain, kept so add_node does not have to walk the whole list
35     node<T>* last;
36
37     /**
38      * Create an empty list
39      */
40     linked_list()
41     {
42         // no nodes yet, so there is no head and no tail
43         first = nullptr;
44         last = nullptr;
45     }
46
47     /**
48      * Add new node to the end of the list
49      *
50      * @param data The data to add
51      */
52     node<T>* add_node(T data)
53     {
54         // declare a pointer for the new node (not yet pointing to valid memory)
55         node<T>* new_node;
56         // allocate a new node on the heap and store its address in new_node
57         new_node = new node<T>();
58
59         // fill in the new node's data
60         new_node->data = data;
61         // this node will be the new tail, so it has no next node yet
62         new_node->next = nullptr;
63
64         // case 1: list is currently empty
65         if (first == nullptr)
66         {
67             // new node is both the first and last node
68             first = new_node;
69             last = new_node;
70         }
71         // case 2: list already has at least one node
72         else
73         {
74             // link the current last node to the new node
75             last->next = new_node;
76             // update last to point to the new tail
77             last = new_node;
78         }
79
80         return new_node;
81     }
82
83     /**
84      * Find and return the node previous to 'target_node'
85      *
86      * @param target_node
87      */
88     node<T>* find_previous_node(node<T>* target_node)
89     {
90         // the head has nothing before it, so there is no previous node
91         if (target_node == first)
92         {
93             return nullptr;
94         }
95
96         // walk the list from the front, one node at a time
97         node<T>* current = first;
98         while (current != nullptr)
99         {
100             // if the node after current is the one we want, current is the answer
101             if (current->next == target_node)
102             {
103                 return current;
104             }
105             // step forward along the chain
106             current = current->next;
107         }
108
109         // reached the end without finding target_node, so it isn't in this list
110         throw string("find_previous_node search failed: node not in list.");
111     }
112
113     /**
114      * Clear all node
115      */
116     void clear()
117     {
118         node<T>* current = first;
119         while (current != nullptr)
120         {
121             // remember the next node BEFORE deleting current,
122             // otherwise current->next is gone and we lose the rest of the list
123             node<T>* next = current->next;
124             // free the heap memory for this node
125             delete current;
126             // move on to the node we saved
127             current = next;
128         }
129         // every node is gone, so the list is now empty
130         first = nullptr;
131         last = nullptr;
132     }
133
134     /**
135      * remove the specific node from the list
136      *
137      * @param del_node
138      */
139     void remove(node<T>* del_node)
140     {
141         node<T>* previous_node = nullptr;
142
143         // case 1: deleting the head, so the second node becomes the new first
144         if (first == del_node)
145         {
146             first = del_node->next;
147             // nothing points to del_node, so there is no previous node
148             previous_node = nullptr;
149         }
150         // case 2: deleting a node in the middle or at the end
151         else
152         {
153             // find the node that currently points to del_node
154             previous_node = find_previous_node(del_node);
155             // unlink del_node by pointing previous straight at del_node's next
156             previous_node->next = del_node->next;
157         }
158
159         // if we removed the tail, the previous node becomes the new tail
160         // (nullptr when the list is now empty)
161         if (last == del_node)
162         {
163             last = previous_node;
164         }
165
166         // now that nothing points to it, free its heap memory
167         delete del_node;
168     }
169
170     /**
171      * Insert a node into a list, after the previous list
172      *
173      * @param in_node
174      * @param previous_node
175      */
176     void insert(node<T>* in_node, node<T>* previous_node)
177     {
178         // the new node takes over whatever previous_node used to point at
179         in_node->next = previous_node->next;
180         // previous_node now points at the new node, splicing it into the chain
181         previous_node->next = in_node;
182
183         // if we inserted after the old tail, the new node is the new tail
184         if (last == previous_node)
185         {
186             last = in_node;
187         }
188     }
189 }

```



```

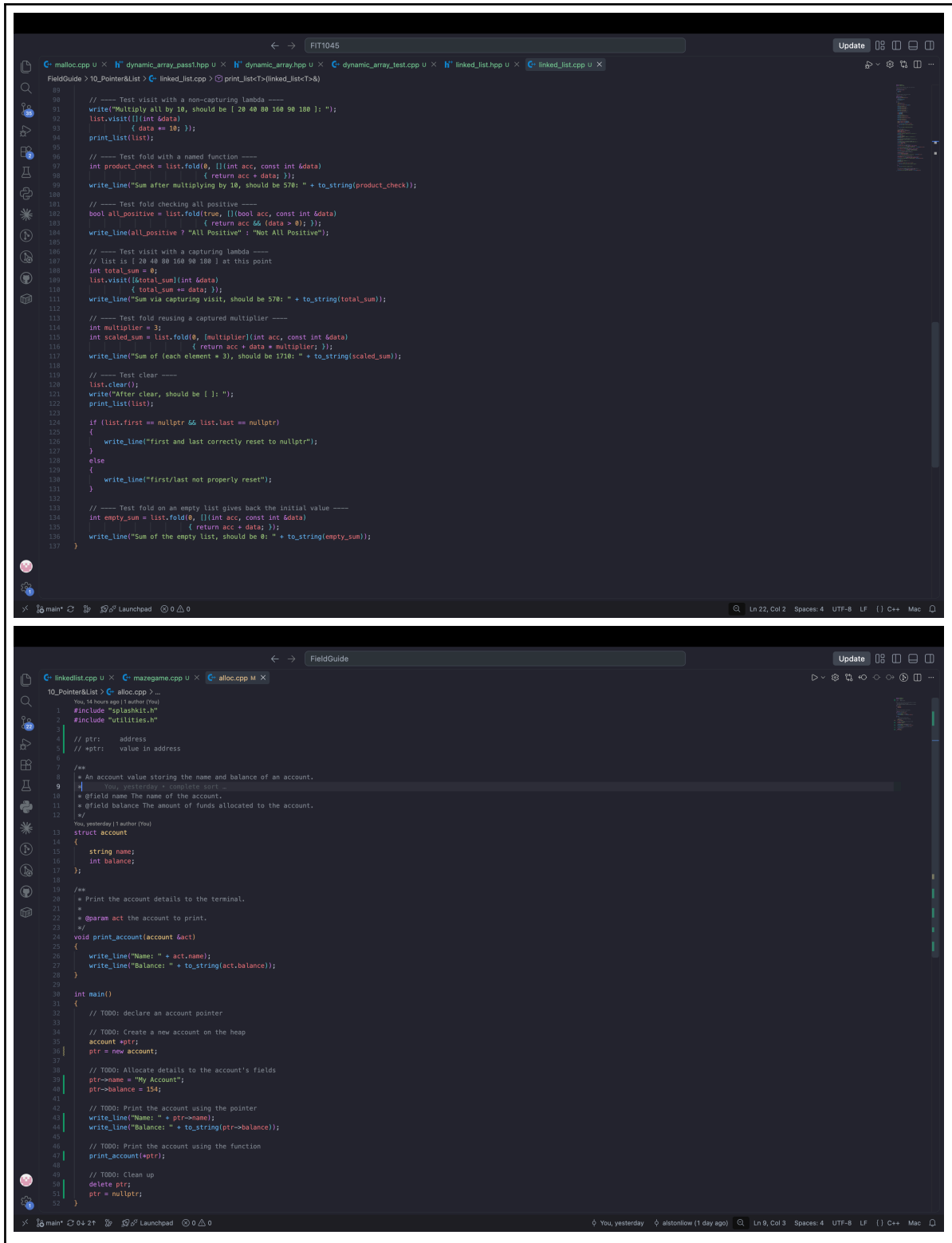
// linked_list.hpp
1 #ifndef LINKED_LIST_HPP
2 #define LINKED_LIST_HPP
3
4 class Linked_List
5 {
6 public:
7     // Insert a new node at the end of the list
8     void insert(Node<T> *in_node, Node<T> *previous_node)
9     {
10         // If we inserted after the old tail, the new node is the new tail
11         if (last == previous_node)
12         {
13             last = in_node;
14         }
15     }
16
17     // Call func once for each value in the list, in order.
18     // func takes the data by reference, so it can read or change it.
19     // @param F any callable (function pointer or lambda)
20     // @param func the action to perform on every element
21     template <typename F>
22     void visit(F func)
23     {
24         // start at the head and walk to the end
25         Node<T> *current = first;
26         while (current != nullptr)
27         {
28             // hand this node's value to the caller's function
29             func(current->data);
30             // step forward along the chain
31             current = current->next;
32         }
33     }
34
35     // Combine every value in the list down to a single result.
36     // func is called as func(running_value, data) and returns the new running value.
37     // @param R the type of the result being built up
38     // @param F any callable (function pointer or lambda)
39     // @param initial_value the starting value before any element is seen
40     // @param func how to fold one element into the running value
41     // @return the running value after the whole list has been folded in
42     template <typename R, typename F>
43     R fold(R initial_value, F func)
44     {
45         // the running result, updated once per node
46         R value = initial_value;
47
48         // walk the list from the head
49         Node<T> *current = first;
50         while (current != nullptr)
51         {
52             // fold this element into the running result
53             value = func(value, current->data);
54             current = current->next;
55         }
56
57         // an empty list just gives back the initial value
58         return value;
59     }
60 };
61
62 #endif

```

```

// linked_list.cpp
1 #include "linked_list.hpp"
2
3 // Print one value followed by a space, for use with visit
4 // @param data the value to print
5 void print_data(int &data)
6 {
7     write(data);
8     write(" ");
9 }
10
11 // Print every value in the list on one line
12 // @param list the list to print
13 template <typename T>
14 void print_list(Linked_List<T> &list)
15 {
16     write("[ ");
17     list.visit(print_data);
18     write_line("]");
19 }
20
21 int main()
22 {
23     Linked_List<int> list;
24
25     // --- Test first node ---
26     Node<int> *first_node = list.add_node(2);
27     write("This should be 2: ");
28     write_line(list.first->data);
29
30     write("This should also be 2: ");
31     write_line(first_node->data);
32
33     // --- Test adding more nodes ---
34     list.add_node(4);
35     list.add_node(8);
36     list.add_node(16);
37
38     write("This should be 4: ");
39     write_line(list.first->next->data);
40
41     write("This should be 8: ");
42     write_line(list.first->next->next->data);
43
44     // --- Test insert ---
45     // Insert 5 between 4 and 8
46     Node<int> *node_five = new Node<int>(5);
47     node_five->next = list.first->next;
48     list.insert(node_five, list.first->next);
49
50     write("After inserting 5, should be [ 2 4 5 8 16 9 18 ]: ");
51     print_list(list);
52
53     // --- Test remove ---
54     list.remove(node_five);
55     write("After removing 5, should be [ 2 4 8 16 9 18 ]: ");
56     print_list(list);
57
58     // --- Test visit with a named function ---
59     write("Visit with print_data, should be [ 2 4 8 16 9 18 ]: ");
60     print_list(list);
61
62     // --- Test visit with a non-capturing lambda ---
63     write("Multiply all by 10, should be [ 20 40 80 160 90 180 ]: ");
64     list.visit([&data](int &data)
65     {
66         data *= 10;
67     });
68     print_list(list);
69
70     // --- Test fold with a named function ---
71     int product_check = list.fold(0, [](int acc, const int &data)
72     {
73         return acc * data;
74     });
75     write_line("Sum after multiplying by 10, should be 5760: " + to_string(product_check));
76
77     // --- Test fold checking all positive ---
78     bool all_positive = list.fold(true, [](bool acc, const int &data)
79     {
80         return acc && (data > 0);
81     });
82     write_line(all_positive ? "All Positive" : "Not All Positive");
83
84     // --- Test visit with a capturing lambda ---
85     // list is [ 20 40 80 160 90 180 ] at this point
86     list.visit([&list, &acc = 1](int &data)
87     {
88         acc *= data;
89     });
89
90     write_line("Sum of all elements: " + to_string(acc));
91 }

```



```

// linked_list.cpp
// ---- Test visit with a non-capturing lambda ----
90 write("Multiply all by 10, should be [ 20 40 80 160 90 180 ]: ");
91 list.visit([&data](int ddata)
92     { data *= 10; });
93 print_list(list);
94
// ---- Test fold with a named function ----
95
96 int product_check = list.fold(0, [](int acc, const int ddata)
97     { return acc + data; });
98 write_line("Sum after multiplying by 10, should be 570: " + to_string(product_check));
99
// ---- Test fold checking all positive ----
100
101 bool all_positive = list.fold(true, [](bool acc, const int ddata)
102     { return acc && (data > 0); });
103 write_line(all_positive ? "All Positive" : "Not All Positive");
104
// ---- Test visit with a capturing lambda ----
105
106 // List is [ 20 40 80 160 90 180 ] at this point
107 int total_sum = 0;
108 list.visit([&total_sum](int ddata)
109     { total_sum += data; });
110 write_line("Sum via capturing visit, should be 570: " + to_string(total_sum));
111
// ---- Test fold reusing a captured multiplier ----
112
113 int multiplier = 3;
114 int scaled_sum = list.fold(0, [multiplier](int acc, const int ddata)
115     { return acc + data * multiplier; });
116 write_line("Sum of each element * 3, should be 1710: " + to_string(scaled_sum));
117
// ---- Test clear ----
118
119 list.clear();
120 write("After clear, should be [ ]: ");
121 print_list(list);
122
if (list.first == nullptr && list.last == nullptr)
123 {
124     write_line("first and last correctly reset to nullptr");
125 }
126 else
127 {
128     write_line("first/last not properly reset");
129 }
130
// ---- Test fold on an empty list gives back the initial value ----
131
132 int empty_sum = list.fold(0, [](int acc, const int ddata)
133     { return acc + data; });
134 write_line("Sum of the empty list, should be 0: " + to_string(empty_sum));
135
136
137

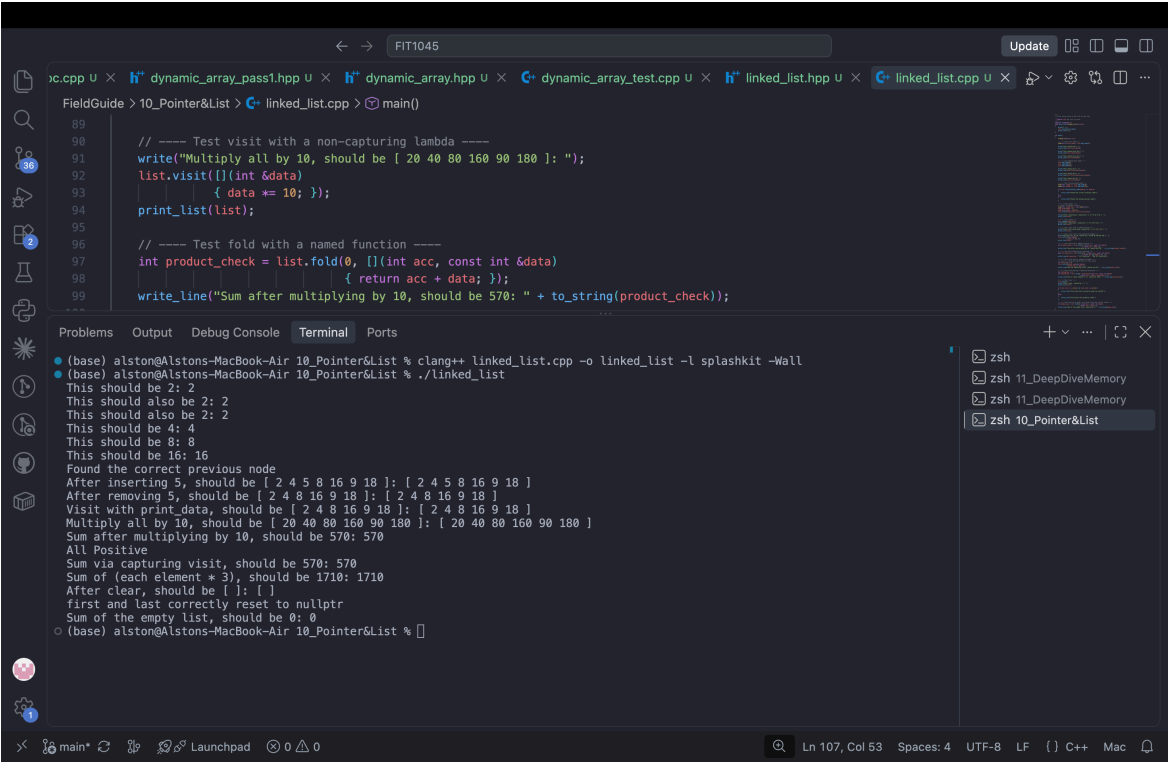
```

```

// mazesgame.cpp
10_Pointer&List > C: alloc.cpp >
1 You, yesterday | 1 author (You)
2 #include "splashkit.h"
3 #include "utilities.h"
4
5 // ptr: address
6 // *ptr: value in address
7
8 /**
9  * An account value storing the name and balance of an account.
10  * You, yesterday | complete sort ...
11  * @field name The name of the account.
12  * @field balance The amount of funds allocated to the account.
13  */
14 struct account
15 {
16     string name;
17     int balance;
18 };
19
20 /**
21  * Print the account details to the terminal.
22  * @param act the account to print.
23  */
24 void print_account(account &act)
25 {
26     write_line("Name: " + act.name);
27     write_line("Balance: " + to_string(act.balance));
28 }
29
30 int main()
31 {
32     // TODO: declare an account pointer
33
34     // TODO: Create a new account on the heap
35     account *ptr;
36     ptr = new account;
37
38     // TODO: Allocate details to the account's fields
39     ptr->name = "My Account";
40     ptr->balance = 154;
41
42     // TODO: Print the account using the pointer
43     write_line("Name: " + ptr->name);
44     write_line("Balance: " + to_string(ptr->balance));
45
46     // TODO: Print the account using the function
47     print_account(*ptr);
48
49     // TODO: Clean up
50     delete ptr;
51     ptr = nullptr;
52 }

```

Run result of the test



```

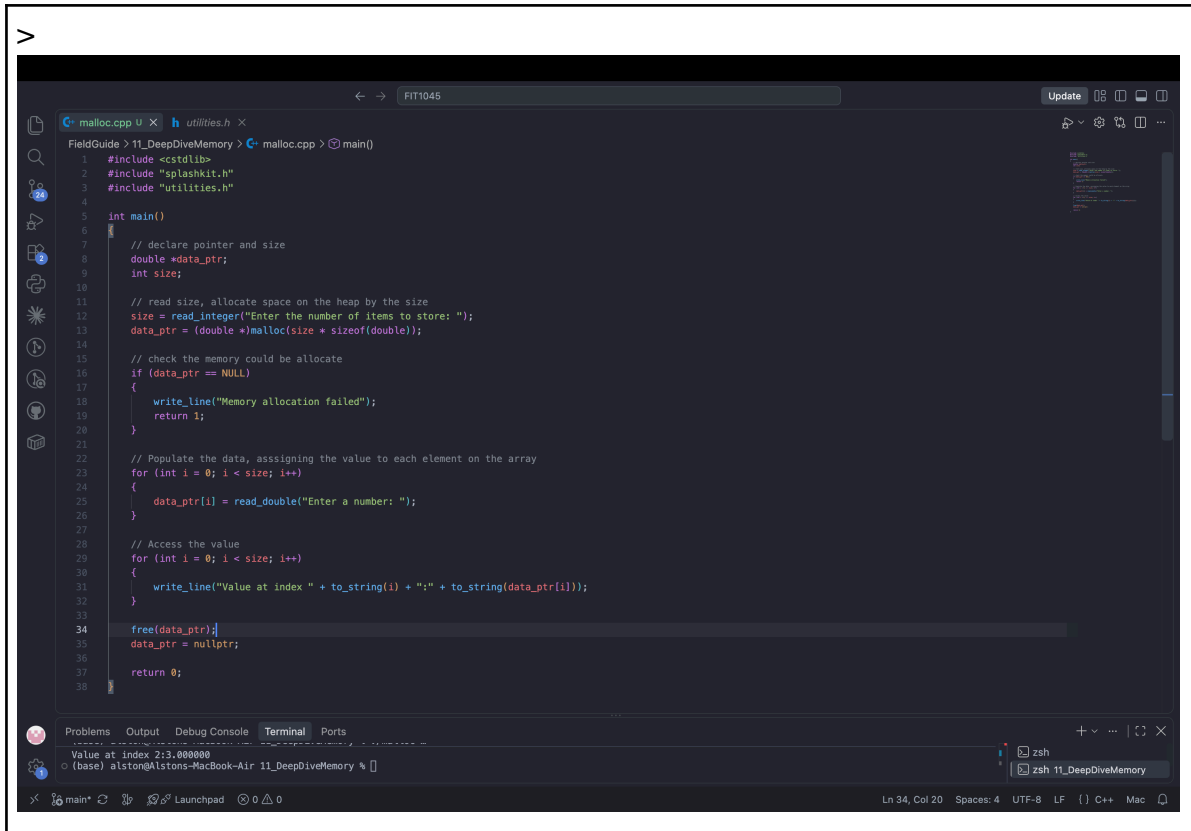
89 // ---- Test visit with a non-capturing lambda ----
90 write("Multiply all by 10, should be [ 20 40 80 160 90 180 ]: ");
91 list.visit([](int &data)
92     { data *= 10; });
93 print_list(list);
94
95 // ---- Test fold with a named function ----
96 int product_check = list.fold(0, [](int acc, const int &data)
97     { return acc + data; });
98 write_line("Sum after multiplying by 10, should be 570: " + to_string(product_check));
99
Problems Output Debug Console Terminal Ports
(base) alston@Alstons-MacBook-Air 10_Pointer&List % clang++ linked_list.cpp -o linked_list -l splashkit -Wall
(base) alston@Alstons-MacBook-Air 10_Pointer&List % ./linked_list
This should be 2: 2
This should also be 2: 2
This should be 4: 4
This should be 8: 8
This should be 16: 16
Found the correct previous node
After inserting 5, should be [ 2 4 5 8 16 9 18 ]: [ 2 4 5 8 16 9 18 ]
After removing 5, should be [ 2 4 8 16 9 18 ]: [ 2 4 8 16 9 18 ]
Visit with print_data, should be [ 2 4 8 16 9 18 ]: [ 2 4 8 16 9 18 ]
Multiply all by 10, should be [ 20 40 80 160 90 180 ]: [ 20 40 80 160 90 180 ]
Sum after multiplying by 10, should be 570: 570
All Positive
Sum via capturing visit, should be 570: 570
Sum of (each element * 3), should be 1710: 1710
After clear, should be [ ]: [ ]
first and last correctly reset to nullptr
Sum of the empty list, should be 0: 0
(base) alston@Alstons-MacBook-Air 10_Pointer&List %
  
```

- Extend your glossary with pages on:
 - Pointers, values, assignment, and pointer operators.
 - New and delete, pointers, and the heap.
 - Linked list (with add, and delete).
- We recommend discussing your progress with the teaching team at this point. No need to submit to OnTrack - just speak with us in class.

Part 3: Learning about malloc

The last piece of the puzzle is `malloc` and `free`. These are lower level C functions for allocating and freeing memory resources.

- Everything you need is in the [Deep Dive Memory](#) chapter of our [Programmer's Field Guide](#).
- Add a screenshot showing you were able to use `malloc` and `free` to create and work with a dynamic array in memory. You can use the [using malloc tour](#).



```

1 #include <cstdlib>
2 #include "splashkit.h"
3 #include "utilities.h"
4
5 int main()
6 {
7     // declare pointer and size
8     double *data_ptr;
9     int size;
10
11     // read size, allocate space on the heap by the size
12     size = read_integer("Enter the number of items to store: ");
13     data_ptr = (double *)malloc(size * sizeof(double));
14
15     // check the memory could be allocate
16     if (data_ptr == NULL)
17     {
18         write_line("Memory allocation failed");
19         return 1;
20     }
21
22     // Populate the data, assigning the value to each element on the array
23     for (int i = 0; i < size; i++)
24     {
25         data_ptr[i] = read_double("Enter a number: ");
26     }
27
28     // Access the value
29     for (int i = 0; i < size; i++)
30     {
31         write_line("Value at index " + to_string(i) + ": " + to_string(data_ptr[i]));
32     }
33
34     free(data_ptr);
35     data_ptr = nullptr;
36
37     return 0;
38 }

```

Problems Output Debug Console Terminal Ports

Value at index 2: 3.000000

(base) alston@Alstons-MacBook-Air 11_DeepDiveMemory %

Ln 34, Col 20 Spaces: 4 UTF-8 LF C++ Mac

- Extend your glossary with pages on malloc and free (and realloc if you can).
- We recommend discussing your progress with the teaching team at this point. No need to submit to OnTrack - just speak with us in class.

Part 4: Working with AI

You now have all of the pieces, so let's work with AI to create a dynamic array.

- Everything you need is in the [Deep Dive Memory](#) chapter of our [Programmer's Field Guide](#).
- Add screenshots showing you working through building a dynamic array with GenAI. You can use the [dynamic array tour](#).

```

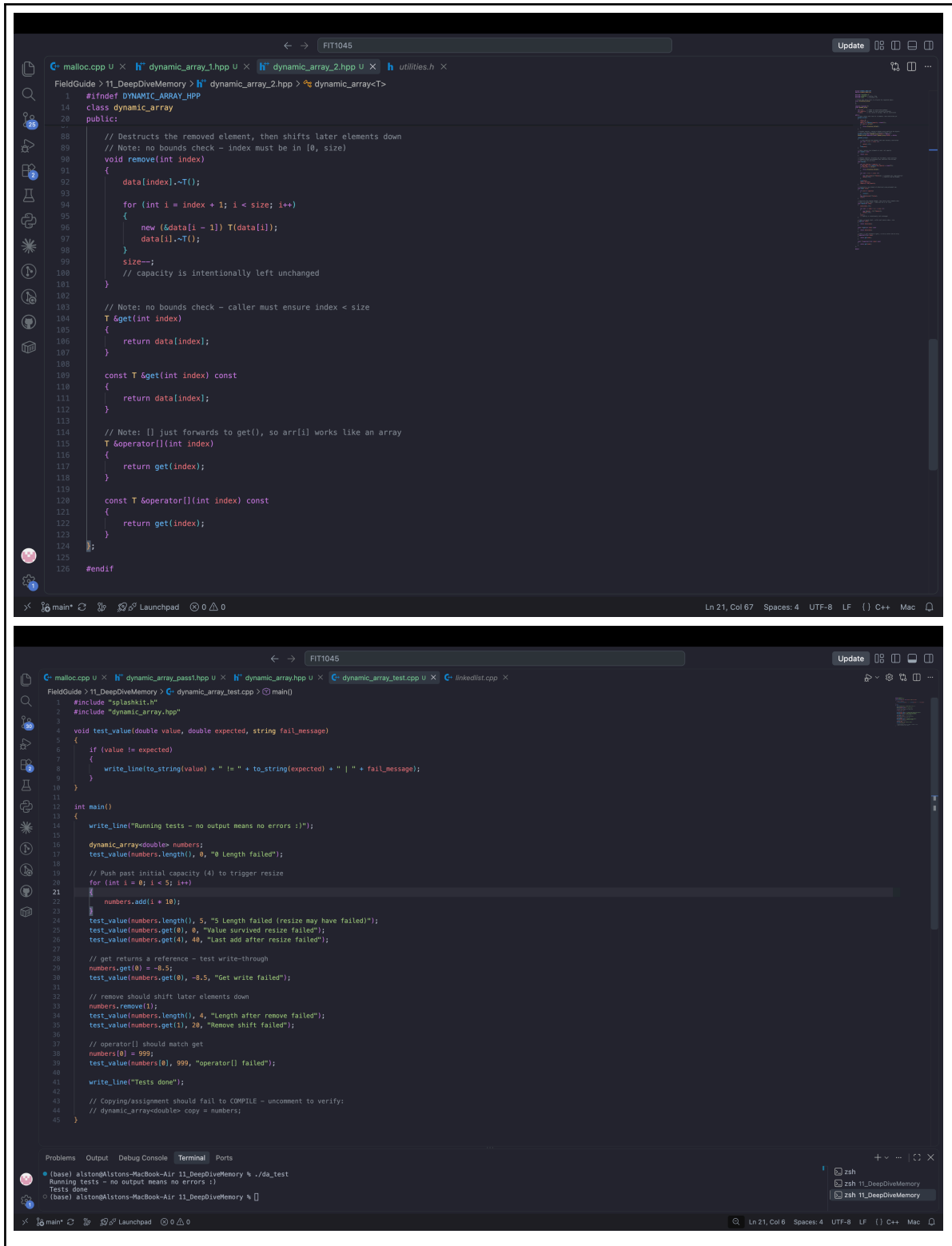
>
C:\main\ > N\dynamic_array_1.hpp u X h utilities.h X
FieldGuide > 11_DeepDiveMemory > N\dynamic_array_1.hpp > dynamic_array<T>
1 #ifndef DYNAMIC_ARRAY_HPP
2 #define DYNAMIC_ARRAY_HPP
3
4 #include "splashkit.h"
5
6 // Grows automatically when capacity is reached (no MAX_CAPACITY template param)
7 template <typename T>
8 class dynamic_array
9 {
10     int size; // elements currently stored
11     int capacity; // elements data can currently hold
12     T *data; // heap storage, resized as needed
13
14 public:
15     // Allocates immediately on creation
16     dynamic_array()
17     {
18         size = 0;
19         capacity = 4;
20         data = new T[capacity];
21     }
22
23     ~dynamic_array()
24     {
25         delete[] data;
26     }
27
28     int length() const
29     {
30         return size;
31     }
32
33     // Doubles capacity, copies old elements across
34     void resize()
35     {
36         int new_capacity = capacity * 2;
37         T *new_data = new T[new_capacity];
38
39         for (int i = 0; i < size; i++)
40         {
41             new_data[i] = data[i];
42         }
43
44         delete[] data;
45         data = new_data;
46         capacity = new_capacity;
47     }
48
49     // Resizes instead of throwing when full
50     void add(T value)
51     {
52         if (size >= capacity)
53         {
54             resize();
55         }
56         data[size] = value;
57         size++;
58     }
59
60     // Capacity never shrinks here
61     void remove(int index)
62     {
63         for (int i = index + 1; i < size; i++)
64         {
65             data[i - 1] = data[i];
66         }
67         size--;
68     }
69
70     T &get(int index)
71     {
72         return data[index];
73     }
74
75     const T &get(int index) const
76     {
77         return data[index];
78     }
79
80     T &operator[](int index)
81     {
82         return get(index);
83     }
84
85     const T &operator[](int index) const
86     {
87         return get(index);
88     }
89
90     // No copy constructor: copying does a shallow pointer copy,
91     // leading to double delete when both objects are destroyed
92 };
93
94 #endif

```

```

>
C:\main\ > N\dynamic_array_1.hpp u X N\dynamic_array_2.hpp u X h utilities.h X
FieldGuide > 11_DeepDiveMemory > N\dynamic_array_2.hpp > ...
1 #ifndef DYNAMIC_ARRAY_HPP
2 #define DYNAMIC_ARRAY_HPP
3
4 #include "splashkit.h"
5 #include <stdlib.h> // malloc, free
6 #include <new> // placement new
7
8 // Throw when malloc fails to allocate the requested memory
9 class allocation_failed
10 {
11 };
12
13 template <typename T>
14 class dynamic_array
15 {
16     int size; // number of constructed elements
17     int capacity; // number of slots currently allocated
18     T *data; // raw, malloc'd storage - NOT all constructed
19
20 public:
21     // Note: starts with room for 4 elements, none constructed yet
22     dynamic_array()
23     {
24         size = 0;
25         capacity = 4;
26         data = (T *)malloc(capacity * sizeof(T));
27         if (data == nullptr)
28         {
29             throw allocation_failed();
30         }
31     }
32
33     // Disable copying - copying a dynamic_array would let two objects
34     // share the same heap memory, leading to double free.
35     dynamic_array(const dynamic_array &other) = delete;
36     dynamic_array &operator=(const dynamic_array &other) = delete;
37
38     ~dynamic_array()
39     {
40         // Only destruct the elements that were actually constructed
41         for (int i = 0; i < size; i++)
42         {
43             data[i].~T();
44         }
45         free(data);
46     }
47
48     // Note: returns size (elements in use), not capacity
49     int length() const
50     {
51         return size;
52     }
53
54     // Doubles capacity: allocates new raw memory, moves existing

```



The image shows two screenshots of a Visual Studio Code editor. The top screenshot displays the implementation of a `dynamic_array` class in `dynamic_array.h`. The bottom screenshot displays the test file `dynamic_array_test.cpp` and the terminal output.

Top Screenshot: `dynamic_array.h`

```

1  #ifndef DYNAMIC_ARRAY_HPP
2  #define DYNAMIC_ARRAY_HPP
3  #include <memory>
4  #include <string>
5  #include <stdexcept>
6  #include <utility>
7
8  namespace fit1045 {
9
10     template <typename T>
11     class dynamic_array {
12     public:
13
14         // Destructs the removed element, then shifts later elements down
15         // Note: no bounds check - index must be in [0, size)
16         void remove(int index) {
17             data[index].~T();
18             for (int i = index + 1; i < size; i++)
19                 new (&data[i - 1]) T(data[i]);
20             data[size].~T();
21             size--;
22             // capacity is intentionally left unchanged
23         }
24
25         // Note: no bounds check - caller must ensure index < size
26         T& get(int index) {
27             return data[index];
28         }
29
30         const T& get(int index) const {
31             return data[index];
32         }
33
34         // Note: [] just forwards to get(), so arr[i] works like an array
35         T& operator[](int index) {
36             return get(index);
37         }
38
39         const T& operator[](int index) const {
40             return get(index);
41         }
42     };
43 #endif

```

Bottom Screenshot: `dynamic_array_test.cpp`

```

1  #include "splatshkit.h"
2  #include "dynamic_array.hpp"
3
4  void test_value(double value, double expected, string fail_message) {
5      if (value != expected) {
6          write_line(to_string(value) + " != " + to_string(expected) + " | " + fail_message);
7      }
8  }
9
10 int main() {
11     write_line("Running tests - no output means no errors :)");
12
13     dynamic_array<double> numbers;
14     test_value(numbers.length(), 0, "0 Length failed");
15
16     // Push past initial capacity (4) to trigger resize
17     for (int i = 0; i < 5; i++)
18         numbers.add(i * 10);
19
20     test_value(numbers.length(), 5, "5 Length failed (resize may have failed)");
21     test_value(numbers.get(0), 0, "Value survived resize failed");
22     test_value(numbers.get(4), 40, "Last add after resize failed");
23
24     // get returns a reference - test write-through
25     numbers.get(0) = -8.5;
26     test_value(numbers.get(0), -8.5, "Get write failed");
27
28     // remove should shift later elements down
29     numbers.remove(1);
30     test_value(numbers.length(), 4, "Length after remove failed");
31     test_value(numbers.get(1), 20, "Remove shift failed");
32
33     // operator[] should match get
34     numbers[0] = 999;
35     test_value(numbers[0], 999, "operator[] failed");
36
37     write_line("Tests done");
38
39     // Copying/assignment should fail to COMPILER - uncomment to verify:
40     // dynamic_array<double> copy = numbers;
41 }

```

Terminal Output:

```

(base) alston@Alstons-MacBook-Air:~/FIT1045/11_DeepDiveMemory % ./da_test
Running tests - no output means no errors :)
Tests done

```

Reflect:

- What was the most interesting thing you learnt through this task, and why?

> The most interesting is about the pointer and memory, especially the pointer. It is a very useful, multifunction and very very abstract concept. It can do many thing but also difficult to understand it and use it very well, I think need many experience to did it well. Not like the past, all is easy to learn, this is very challenging

- What challenges and opportunities do you see with dynamic memory management?

> The manual memory management is very challenging, difficult and abstract. It is very dangerous because we have freedom to control the memory, but it will make many bug we cannot find it. We cannot see the actual thing in the heap, so when we allocate by malloc, maybe will cause some memory issue.

I learned about the actual theory and knowledge about memory. Now I know that the memory can be controlled very precise manually, not just rely on the programming language (or compiler/ computer/ operating system) . And I know that how actually the low-level of computer system control the memory.

- Briefly reflect on the use of GenAI in this task. How helpful was it, and what aspects did you need to be on top of in order to direct it effectively?

> When using AI, the most important thing is we must be the designer. We must know that what we want, so AI can give us the correct version, not the simply and not accurate one. AI can help us to do many repeat work and also give us some idea when we stuck, can be a helper, but not replace us.

We must able to read the thing AI give us, we need to have the basic knowledge, and also keep learning when face the thing we don't know, not just let AI do everything.

We also must know how to give a good prompt, a good specification, this need us to think the requirement, not just simply give the instruction and then hope AI can do everything for us. We must clearly know the specification and what we want.

- Looking back, how long did it take to complete this task? What did you spend the most time on?

> 15 hours. I spent most time in memory and pointer part, this two new concept is close connection, have many syntax, many ways to use, useful. So it is many content need to learn, think and to digest.

Disclose:

- Explain your use of Generative AI for this task. If you didn't use it, just write "I did not use any AI for this task". If you did use it, please state which AI model you used (e.g. ChatGPT) and how.

> I use Claude. I use to understand the new concept include template, throw, try, catch, build the class for the error. And also about the memory, pointer, malloc, calloc, realloc.

Submit:

- Export this worksheet as a PDF.
- Upload your submission to OnTrack:

-